

SYSTÈME ET RÉSEAUX : PARTIEL 2

mai 2025 — Pierre Rousselin

Consignes et conseils :

- Les prototypes des appels système et de certaines fonctions sont rappelés en fin d'énoncé.
- Pour gagner du temps, sauf indication contraire, vous pourrez supposer que tous les appels système réussissent, sauf ceux qu'il faut impérativement tester car bien trop susceptibles d'échouer.
- Les inclusions de fichiers d'en-tête ne sont pas nécessaires non plus.
- Le C dont il est question dans cet énoncé est le C au standard 99, privé des VLA (*Variable Length Arrays*), avec les ajouts du standard POSIX.
- Il est inutile de recopier l'énoncé. Lorsque vous devez compléter des programmes, il suffit d'indiquer clairement à quel endroit (numéro de ligne, corps de fonction...) vous ajoutez des instructions ou modifiez le programme.
- La clarté et la simplicité des solutions seront des éléments importants de l'évaluation.
- Le total des points avant changement de barème éventuel est 64. La note sera ramenée sur 20.
- Le barème est donné à titre indicatif et est susceptible d'être modifié.

Exercice 1 : *threads* et parallélisation (13 points)

On suppose données les fonctions suivantes, qu'on ne peut pas modifier :

```
double f1(double x);
double f2(double x);
double f3(double x, double y);
double f4(double x, double y);
double f5(double x, double y);
```

Le programme suivant demande un nombre à l'utilisateur puis affiche le résultat d'un calcul qui utilise les fonctions précédentes.

```
int main(void)
{
    double x;
    scanf("%lg", &x);
    double res = f5(f4(f1(x), f2(x)), f3(f1(x), f2(x)));
    printf("résultat : %g\n", res);
}
```

Les fonctions *f1*, ..., *f5* se terminent et retournent bien un résultat valide. C'est la seule hypothèse qu'on fera sur ces fonctions.

1. (1 point) Représenter les dépendances entre les appels de fonctions *f1*, *f2*,
2. (12 points) Paralléliser au maximum ce calcul en créant deux nouveaux *threads* et en respectant les consignes suivantes (mais une solution qui ne respecterait pas toutes ces consignes sera tout de même évaluée) :
 - Le *thread* principal ne fait aucun calcul.
 - Les *threads* qui calculent récupèrent la valeur de *x* en utilisant l'argument *arg* de *pthread_create*.
 - Les résultats des appels des *f1* et *f2*, et seulement ceux-ci, sont dans deux variables globales *x1* et *x2*.
 - Lorsqu'un *thread* a terminé l'appel de *f1* ou l'appel de *f2*, il affiche le résultat sur le terminal, par exemple :


```
f1(x) = 3.5
```

 mais les affichages sur le terminal ne sont pas supposés atomiques. Utiliser un mécanisme vu en cours pour que ces affichages ne soient jamais entremêlés.

- Les sémaphores et/ou mutex que vous utilisez peuvent être globaux, tout comme les variables `x1` et `x2` mentionnées précédemment, mais c'est tout.
- Le *thread* qui appelle la fonction `f5` doit récupérer la *valeur de retour* de l'autre *thread* qui allouera sur le tas de quoi stocker un **double**.
- Le *thread* principal récupère de même la valeur de retour du *thread* ayant appelé `f5` et affiche le résultat.
- S'il le faut, introduire des types structurés.
- Toute la mémoire allouée dynamiquement est libérée.

--- * ---

Correction de l'exercice 1 :

- 1.
2. Une solution parmi d'autres (par exemple quelques étudiants ont intelligemment lancé `th1` dans `th2`, ce qui permettait de ne pas définir de structure).

```
void *thread1(void *arg);
void *thread2(void *arg);

double x1, x2;
sem_t f1fini, f2fini;
pthread_mutex_t stdout_lock;
struct argth {
    double x;
    pthread_t th;
};
int main(void)
{
    double x;
    pthread_mutex_init(&stdout_lock, NULL);
    sem_init(&f1fini, 0, 0);
    sem_init(&f2fini, 0, 0);
    scanf("%lg", &x);
    pthread_t th1, th2;
    pthread_create(&th1, NULL, thread1, &x);
    struct argth arg = { .x = x, .th = th1 };
    pthread_create(&th2, NULL, thread2, &arg);
    void *res;
    pthread_join(th2, &res);
    double x5 = *((double *) res);
    free(res);
    printf("résultat : %g\n", x5);
}

void *thread1(void *arg)
{
    double x = *((double *) arg);
    x2 = f2(x);
    sem_post(&f2fini);
    pthread_mutex_lock(&stdout_lock);
    printf("f2(x) = %g\n", x2);
    pthread_mutex_unlock(&stdout_lock);
    sem_wait(&f1fini);
}
```

```

    double *res = malloc(sizeof(double));
    *res = f3(x1, x2);
    return res;
}

void *thread2(void *arg)
{
    struct argth *a = arg;
    x1 = f1(a->x);
    sem_post(&f1fini);
    pthread_mutex_lock(&stdout_lock);
    printf("f1(x) = %g\n", x2);
    pthread_mutex_unlock(&stdout_lock);
    sem_wait(&f2fini);
    double x4 = f4(x1, x2);
    double *res = malloc(sizeof(double));
    void *x3_void;
    pthread_join(a->th, &x3_void);
    double x3 = *((double *) x3_void);
    free(x3_void);
    *res = f5(x3, x4);
    return res;
}

--- * ---

```

Exercice 2 : L'oracle aux 12 singes (14 points)

On suppose données les fonctions suivantes qui sont des boîtes noires qu'on ne peut pas modifier.

```

#define MAX 9000
/** Pour toujours, essayer de deviner le secret de l'univers, compris entre
 * 0 et MAX. À chaque tentative, l'écrire avec printf en décimal sur la sortie
 * standard en écrivant 4 chiffres décimaux (par exemple, pour 130, écrit 0130)
 * suivis d'une fin de ligne. */
void singe(void);
/** Retourner 1 si nombre est bien le secret de l'univers, 0 sinon. */
int verifier(int nombre);

```

La fonction main du programme commence de la façon suivante :

```

int main(void)
{
    setvbuf(stdout, NULL, _IOLBF, 0);
    /* ... */
}

```

de sorte que toutes les écritures dans la sortie standard qui utilisent les fonctions de `stdio.h` sont *line-buffered*, c'est-à-dire ont lieu lorsque les processus écrivent des lignes. Il est inutile de réécrire cet appel à `setvbuf` par la suite.

- (2 points) Écrire la définition de la fonction suivante en utilisant l'appel système `read` et en appelant la fonction `verifier`. Tenir compte de la forme des écritures dans la fonction `singe`. On pourra utiliser la fonction `sscanf` ou la fonction `atoi`.

```

/** Lire le prochain entier dans le fichier de descripteur fd
 * retourner 1 si l'entier est le secret de l'univers, 0 sinon. */
int verifier_read(int fd);

```

2. (3 points) Cela prend beaucoup moins de temps pour l'oracle de vérifier que pour les singes de proposer un entier. L'oracle a donc l'idée de créer 12 singes dans 12 processus enfants qui écrivent dans 12 tubes. L'oracle est le processus parent et vérifie les entiers générés par les singes. Écrire dans une fonction `main`, le code permettant de créer 12 tubes dans un tableau

```
int tubes[12][2];
```

et de créer 12 enfants, dont les PIDs sont stockés (chez le parent) dans un tableau

```
int pids[12];
```

la sortie du processus de PID `pids[i]` étant redirigée dans la partie écriture de `tubes[i]`. Vous ferez aussi les autres opérations sur `tubes` qui sont nécessaires. Chaque enfant exécute ensuite la fonction `singe`.
3. (1 point) Quel problème se pose maintenant à l'oracle aux 12 tubes? Quel appel système utiliser pour y remédier?
4. (5 points) Écrire la suite du programme (dans `main` ou des fonctions séparées, au choix) : tant que l'oracle n'a pas trouvé le secret de l'univers, il attend qu'un des tubes soit prêt pour la lecture puis vérifie s'il contient bien le secret de l'univers. Si c'est le cas, il affiche le secret de l'univers et retient l'indice associé au singe.
5. (3 points) Enfin, l'oracle décide de tuer tous les singes, à l'exception de celui qui a trouvé le secret de l'univers. Décidant qu'il faut maintenant et pour toujours porter la parole du singe à tous, il exécute le programme `cat` après avoir redirigé son entrée sur le tube dans lequel écrit le singe devin. Écrire les lignes correspondant à cette situation dans la fonction `main`.

--- * ---

Correction de l'exercice 2 :

1.

```
int verifier_read(int fd)
{
    char buf[6];
    read(fd, buf, 5);
    buf[5] = '\0';
    int reponse;
    sscanf(buf, "%d", &reponse);
    return verifier(reponse);
}
```
2.

```
int tubes[NUM_SINGES][2], pids_singes[NUM_SINGES];
int i;
for (i = 0; i < NUM_SINGES; i++) {
    pipe(tubes[i]);
    if ((pids_singes[i] = fork()) == 0) {
        close(tubes[i][0]);
        dup2(tubes[i][1], 1);
        setvbuf(stdout, NULL, _IOLBF, 0);
        singe();
        exit(0);
    }
    close(tubes[i][1]);
}
```
3. L'oracle ne peut pas simplement attendre avec `read` dans un tube qu'un nombre arrive car d'autres nombres pourraient arriver dans les autres tubes. Il peut utiliser l'appel système `poll` pour attendre qu'un des tubes soit prêt pour la lecture.
4.

```
int oracle(int tubes[][2], int nb_singes)
{

```

```

    struct pollfd *fds = malloc(nb_singes * sizeof(struct pollfd));
    int i;
    for (i = 0; i < nb_singes; i++) {
        fds[i].fd = tubes[i][0];
        fds[i].events = POLLIN;
    }
    int trouve = 0;
    int bon_singe;
    while (!trouve) {
        poll(fds, nb_singes, -1);
        for (i = 0; i < nb_singes; i++) {
            if (fds[i].revents & (POLLIN | POLLHUP)) {
                if (verifier_read(fds[i].fd)) {
                    trouve = 1;
                    bon_singe = i;
                }
            }
        }
    }
    free(fds);
    return bon_singe;
}

```

5. À la suite de la fonction main :

```

    int bon_singe = oracle(tubes, NUM_SINGES);
    for (i = 0; i < NUM_SINGES; i++) {
        if (i != bon_singe) {
            kill(pids_singes[i], SIGTERM);
        }
    }
    printf("Le bon singe est %d\n", bon_singe);
    dup2(tubes[bon_singe][0], 0);
    execlp("cat", "cat", NULL);

```

--- * ---

Exercice 3 : Éléments d'un client HTTP (37 points)

Le protocole HTTP (*hypertext transfer protocole*) a pour principal but que des clients (en général, des navigateurs web) fassent des requêtes à des serveurs HTTP (comme `apache` ou `nginx`) qui, en cas de succès, leur transmettent des contenus (par exemple un fichier `html`).

Nous parlons dans cet exercice des versions 1.0 et 1.1 de ce protocole, sans sa couche sécurisée. Un serveur HTTP écoute les demandes de connexion en TCP sur le port 80.

1. (1 point) Que signifie TCP et quel est son rôle ? Qui est responsable de sa mise en œuvre ?
2. (1 point) Qu'est-ce qu'un numéro de port (valeurs possibles, boutisme) et à quoi cela sert-il ?
3. (3 points) Écrire le code en C permettant à un serveur d'écouter ces demandes de connexion sur le port 80 et d'accepter de telles demandes.

Le client que nous allons écrire est très simple, voici son mode d'emploi.

```

Usage: ./httpclt nom[/ressource]
client HTTP qui écrit un contenu dans le terminal
nom est un nom de domaine
Exemple: ./httpclt www.perdu.com
Le nom est suivi d'une ressource éventuelle comme :
Exemple: ./httpclt www.linuxfocus.org/English/Archives/

```

4. (2 points) On suppose donnée une fonction

```
void usage(char *nom_prog);
```

qui affiche le mode d'emploi précédent sur la sortie standard des erreurs en utilisant `nom_prog` comme nom de programme (à la place d'écrire en dur `./httpclt`). Écrire le début d'une fonction `main` qui prend des arguments et

- si le nombre d'arguments du processus (sans compter le nom de commande) n'est pas 1, affiche le mode d'emploi précédent et se termine avec un statut de sortie correspondant à un échec ;
 - s'il y a un argument, renseigne les deux chaînes de caractères `nom` et `ressource` correspondantes. Si la ressource n'est pas donnée dans l'argument, y mettre simplement la chaîne `"/"`.
5. (1 point) Comment se nomme (sigle et nom complet) le procédé qui permet d'associer une ou plusieurs adresses IP à un nom de domaine ? Quel protocole de la couche transport utilise-t-il le plus souvent ?
6. (4 points) Écrire la définition de la fonction suivante :

```
/** Essayer de se connecter en TCP à un serveur de nom name au service service
 * name et service sont des chaînes terminées par un '\0'
 * name contient un nom de domaine
 * service peut contenir un numéro de port en décimal ou le nom d'un service comme http */
int create_connected_sock(char *name, char *service);
```

puis en faire un appel correct dans la fonction `main` commencée précédemment. ***Vous pourrez avoir une partie des points en répondant partiellement à cette question ou en supposant qu'une adresse IPv4 est donnée dans `name`.***

Les requêtes HTTP sont formées de lignes terminées par CRLF ("`\r\n`"). Une ligne vide met fin à la requête. Nous ne nous intéressons ici qu'à la forme la plus simple de requête GET, qui permet de demander une ressource chez un hôte. Par exemple, pour obtenir la page d'accueil de `www.perdu.com`, une requête possible est :

```
GET / HTTP/1.1
Host: www.perdu.com
```

Remarquer le `/` pour la racine, la page d'accueil (souvent `index.html`) et le fait qu'on rappelle le nom de domaine derrière l'attribut `Host` (parce qu'il est courant qu'un même serveur serve plusieurs sites). Pour demander la page `www.linuxfocus.org/English/Archives/`, on utilise la requête :

```
GET /English/Archives/ HTTP/1.1
Host: www.linuxfocus.org
```

7. (2 points) Écrire la suite de la fonction `main` en envoyant au serveur la requête GET correspondant à l'argument de la ligne de commande.

Après avoir reçu la requête du client, le serveur va envoyer un nombre variable de lignes, toutes terminées par CRLF. ***Il n'y a aucune limite à la taille des lignes, il n'y a pas non plus de limite au nombre de telles lignes.***

8. (4 points) Écrire la définition de la fonction suivante :

```
/** Lire dans le fichier de descripteur fd, jusqu'aux deux caractères "\r\n"
 * compris. La lecture se fait octet par octet pour des raisons de simplicité
 * Retourne un pointeur vers une chaîne dynamiquement allouée contenant la ligne
 * et terminée par un '\0'; cette ligne doit être libérée par la suite avec free
 * En cas d'erreur, ou si la fin du fichier a été rencontrée, retourne NULL */
char *read_until_crlf(int fd);
```

On pourra se servir de la fonction `realloc`, dont voici le manuel :

```
void *realloc(void *ptr, size_t size);
```

La fonction `realloc()` modifie la taille du bloc de mémoire pointé par `ptr` à la taille de `size` octets. Le contenu de la mémoire entre la zone de départ et la plus petite taille entre la nouvelle et l'ancienne taille n'est pas modifié. Si la zone pointée était déplacée, un `free(ptr)` est effectué. Retourne un pointeur vers la mémoire allouée ou `NULL` en cas d'échec.

La première des lignes envoyées par le serveur contient la version de HTTP proposée (qui, en général, devrait être la même que celle envoyée par le client), puis un code de 3 chiffres décimaux indiquant peut-être une erreur, suivi d'une chaîne de caractères qui peut apporter des précisions.

Par exemple, la ligne `HTTP/1.1 200 OK` indique que tout va bien. Mais la ligne (bien connue) `HTTP/1.1 404 Not Found` indique que la ressource demandée n'est pas présente sur le serveur. Seul le code 200 indique un succès.

9. (2 points) Écrire la fonction

```
/** Lire la première ligne de la réponse du serveur à une requête HTTP depuis
 * le descripteur de fichier fd.
 * Si le serveur signale un échec, le code et le message d'erreur correspondants
 * sont écrits sur la sortie des erreurs.
 * Retourne 0 en cas de succès, le code d'erreur correspondant en cas d'échec */
int get_http_status(int fd);
```

Les lignes qui suivent sont des *en-têtes* HTTP. Une ligne vide signale la fin de ces en-têtes. Les en-têtes ont en général la forme **Attribut: valeur**, par exemple :

```
Content-Type: text/html
Date: Thu, 22 May 2025 14:20:45 GMT
Content-Length: 1256
Connection: keep-alive
```

10. (3 points) Définir la fonction suivante :

```
/** Lire les en-têtes d'une réponse HTTP depuis le descripteur de fichier fd
 * Retourne un pointeur vers un tableau dynamiquement alloué de pointeurs vers
 * les chaînes composant l'en-tête, privées de leurs caractères "\r\n".
 * La ligne vide servant à délimiter la fin de l'en-tête ne figure pas dans le tableau.
 * La dernière case du tableau contient la valeur NULL.
 * Le tableau retourné, ainsi que chacun de ses éléments doivent être libérés par
 * la suite avec free. */
char **get_headers(int fd);
```

11. (1 point) Écrire les instructions dans `main` permettant d'appeler correctement ces fonctions.

Nous nous intéressons uniquement à trois en-têtes qui nous renseignent sur la façon dont le serveur va envoyer le contenu qui suivra les en-têtes.

Content-Length: <number> le serveur annonce que le contenu fait <number> octets, où <number> est un entier positif écrit en décimal; il enverra ces octets juste après la fin des en-têtes

Connection: close le serveur indique qu'il fermera la connexion TCP à la fin de la transmission

Transfer-Encoding: chunked le serveur va envoyer le contenu par morceaux. Avant chaque morceau, il envoie une ligne (terminée par CRLF) contenant uniquement la taille, *en hexadécimal* du prochain morceau; il envoie ensuite le morceau, puis juste après, une ligne vide. Le dernier morceau et seulement celui-ci a une taille égale à 0.

On suppose, pour simplifier, que seule une de ces 3 en-têtes est présente. *Attention : les en-têtes HTTP ne sont pas sensibles à la casse* donc par exemple, `CONTENT-LENGTH` et `Content-Length` signifient la même chose. La fonction POSIX `strcasecmp` compare deux chaînes, comme `strcmp`, mais sans tenir compte de la casse.

12. (3 points) Définir la fonction suivante :

```
#define FTM_ERR -3
#define CHUNKED -2
#define CONNCLO -1
/** Lit le tableau d'en-têtes (voir get_headers pour le format du tableau) pour
 * déterminer la méthode de transfert du fichier.
 * Retourne :
 * CHUNKED si l'en-tête contient Transfer-Encoding: chunked
 * CONNCLO si l'en-tête contient Connection: close
 * un entier >= 0 égal à la valeur de l'en-tête de nom Content-Length si elle
 * est présente
 * FTM_ERR si aucune de ces 3 en-têtes n'est présente. */
long content_transfer_method(char **headers);
```

13. (2 points) Définir la fonction suivante :

```
/** Obtenir depuis le fichier décrit par fd et afficher sur la sortie standard
 * le contenu qui suit, en supposant que la méthode de transfert est
 * "Connection: close" */
void get_content_close(int fd);
```

14. (3 points) Définir les deux fonctions suivantes. La première doit utiliser un *buffer*, disons de 1024 octets. Faire attention à ne pas lire plus de *nbytes* au total et à ne pas lire plus de 1024 octets d'un seul coup.

```
/** Lire exactement nbytes dans le fichier fdin et les écrire dans fdout
 * S'arrête lorsque nbytes ont été lus ou que la fin du fichier fdin est atteinte. */
void read_write_nbytes(int fdin, int fdout, size_t nbytes);
/** Obtenir depuis le fichier décrit par fd et afficher sur la sortie standard
 * le contenu qui suit, en supposant que la méthode de transfert est
 * "Content-Length" et que la valeur associée est length */
void get_content_ctlen(int fd, long length);
```

15. (3 points) Définir la fonction suivante :

```
/** Obtenir depuis le fichier décrit par fd et afficher sur la sortie standard
 * le contenu qui suit, en supposant que la méthode de transfert est
 * "Transfer-Encoding: chunked" */
void get_content_chunked(int fd);
```

16. (2 points) Terminer l'écriture de la fonction `main` avec les appels manquants et la libération de toutes les ressources.

--- * ---

Correction de l'exercice 3 :

1. TCP signifie *Transmission Control Protocol*. Comme tous les protocoles de la couche transport, il est mis en œuvre par les systèmes d'exploitations. Il permet un échange de flux de données fiable.
2. Un numéro de port est un entier sur deux octets en *Network Byte Order* (gros-boutisme). Il est donc compris entre 0 et 65535. Un numéro de port sert à transmettre les données au bon processus.
3.

```
int sock = socket(AF_INET, SOCK_STREAM, 0);
if (sock < 0)
    perror_exit("socket");
struct sockaddr_in sa = { .sin_family = AF_INET,
    .sin_port = htons(80),
    .sin_addr = { htonl(INADDR_ANY) } };
if (bind(sock, (struct sockaddr *) &sa, sizeof(sa)) < 0) {
    perror("bind");
```



```

        exit(2);
    }
    if (listen(sock, 128) < 0) {
        perror("listen");
        return -1;
    }
    for (;;) {
        int s = accept(sock, NULL, NULL);
        /* ... */
    }
}

4. int main(int argc, char *argv[])
{
    if (argc != 2) {
        usage(argv[0]);
        return 1;
    }
    char *nom = argv[1];
    char *ressource;
    if (strchr(nom, '/')) {
        ressource = strdup(strchr(nom, '/'));
        *strchr(nom, '/') = '\0';
    } else {
        ressource = strdup("/"); /* to make consistent frees */
    }
    /* ... */
}

5. Le DNS (Domain Name System) permet d'associer des adresses IP à des noms de domaines. Il est
    en général au-dessus d'UDP.

6. int create_connected_sock(char *name, char *service)
{
    struct addrinfo hints = { 0 }, *res, *iter;
    int sock;
    int stat = getaddrinfo(name, service, &hints, &res);
    if (stat != 0) {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(stat));
        exit(1);
    }
    for (iter = res; iter != NULL; iter = iter->ai_next) {
        sock = socket(iter->ai_family, iter->ai_socktype, iter->ai_protocol);
        if (sock < 0) {
            perror("sock");
            continue;
        }
        if (connect(sock, iter->ai_addr, iter->ai_addrlen) == 0) {
            break;
        }
        close(sock);
        sock = -1;
    }
    freeaddrinfo(res);
    return sock;
}

```

Puis, dans main

```

int sock = create_connected_sock(argv[1], "http");
7. dprintf(sock, "GET %s HTTP/1.1\r\n", ressource);
   dprintf(sock, "Host: %s\r\n\r\n", nom);
8. char *read_until_crlf(int fd)
   {
       size_t res_sz = 128, tot = 0;
       char *res = malloc(res_sz);
       ssize_t n;
       do {
           n = read(fd, res + tot, 1);
           if (n <= 0) {
               free(res);
               return NULL;
           }
           tot += n;
           if (tot == res_sz - 1) {
               res_sz *= 2;
               res = realloc(res, res_sz);
           }
       } while (tot < 2 || res[tot - 2] != '\r' || res[tot - 1] != '\n');
       res[tot] = '\0';
       return res;
   }
9. int get_http_status(int fd)
   {
       char *line = read_until_crlf(fd);
       line[strlen(line) - 2] = '\0';
       char httpvers[16];
       char errcode[4];
       sscanf(line, "%s %s", httpvers, errcode);
       int stat = atoi(errcode);
       if (stat != 200) {
           fprintf(stderr, "%s\n", strstr(line, errcode));
       } else {
           stat = 0;
       }
       return stat;
   }
10. char **get_headers(int fd)
    {
        size_t hdr_sz = 16, hdr_len = 0;
        char **headers = malloc(hdr_sz * sizeof(char *));
        char *line;
        do {
            line = read_until_crlf(fd);
            printf("%s", line);
            headers[hdr_len++] = line;
            if (hdr_len == hdr_sz) {
                hdr_sz *= 2;
                headers = realloc(headers, hdr_sz * sizeof(char *));
            }
        }
    }

```

```

        line[strlen(line) - 2] = '\0';
    } while (strlen(line) > 0);
    free(line);
    headers[hdr_len - 1] = NULL;
    return headers;
}

11. get_http_status(sock);
    char **headers = get_headers(sock);
12. long content_transfer_method(char **headers)
    {
        for (size_t i = 0; headers[i] != NULL; i++) {
            if (!strcasecmp(headers[i], "Transfer-Encoding: chunked"))
                return CHUNKED;
            if (!strcasecmp(headers[i], "Connection: close"))
                return CONNCLO;
            char attr[64];
            size_t val;
            if (sscanf(headers[i], "%63[^:]: %zu", attr, &val) == 2) {
                if (!strcasecmp(attr, "Content-Length"))
                    return val;
            }
        }
        return CTM_ERR;
    }

13. void get_content_close(int fd)
    {
        char buf[1024];
        ssize_t n;
        while ((n = read(fd, buf, sizeof(buf))) > 0) {
            write(1, buf, n);
        }
    }

14. void read_write_nbytes(int fdin, int fdout, size_t nbytes)
    {
        ssize_t r;
        size_t tot = 0;
        char buf[1024];
        while (tot < nbytes) {
            size_t max = nbytes - tot < 1024 ? nbytes - tot : 1024;
            r = read(fdin, buf, max);
            if (r == 0) {
                return;
            }
            write(fdout, buf, r);
            tot += r;
        }
    }

    void get_content_ctlen(int fd, long length)
    {
        read_write_nbytes(fd, 1, length);
    }

```

```

15. void get_content_chunked(int fd)
{
    char *line;
    size_t chunksz;
    do {
        line = read_until_crlf(fd);
        sscanf(line, "%zx", &chunksz);
        free(line);
        read_write_nbytes(fd, 1, chunksz);
        /* Ligne vide après chaque chunk */
        line = read_until_crlf(fd);
        free(line);

        } while (chunksz > 0);
    }

16. long n = content_transfer_method(headers);
    if (transfer_method == CHUNKED) {
        get_content_chunked(fd);
    } else if (transfer_method == CONNCL0) {
        get_content_close(fd);
    } else if (transfer_method >= 0) {
        get_content_ctlen(fd, transfer_method);
    } else {
        fprintf(stderr, "Invalid transfer method %ld\n", transfer_method);
    }
    close(sock);
    int i;
    for (i = 0; headers[i] != NULL; i++)
        free(headers[i]);
    free(headers);
    free(ressource);

    --- * ---

```

Prototypes des fonctions et appels systèmes à utiliser

Certains arguments sont mis à NULL ou 0 car nous n'avons pas vu leurs rôles en cours.

```

/* fonctions de stdlib.h */
int atoi(const char *nptr);
void *malloc(size_t taille);
void free(void *ptr);
void *realloc(void *ptr, size_t taille);

/* fonctions de string.h */
size_t strlen(const char *s);
char *strcpy(char *dest, const char *src);
char *strcat(char *dest, const char *src);
/* retourne un pointeur vers la première occurrence de aiguille dans
   meule_de_foin ou NULL si elle n'y figure pas */
char *strstr(char *meule_de_foin, char *aiguille);
/* retourne une copie de s allouée sur le tas */
char *strdup(const char *s);

```

```

/* retourne un pointeur vers la première occurrence du caractère c dans la
   chaîne s, ou NULL si elle n'y figure pas. */
char *strchr(const char *s, int c);
int strcasecmp(const char *s1, const char *s2);
int strcmp(const char *s1, const char *s2);
void *memcpy(void *dest, const void *src, size_t n);
void *memset(void *s, int c, size_t n);

/* fonctions de stdio.h */
int getchar(void);
int putchar(int c);
int printf(const char *format, ...);
int fprintf(FILE *flux, const char *format, ...);
int dprintf(int fd, const char *format, ...);
int sprintf(char *chaîne, const char *format, ...);
int sscanf(const char *str, const char *restrict format, ...);

/* fonctions de gestion des processus */
pid_t fork(void);
pid_t wait(int *wstatus);
void exit(int status);
int kill(pid_t pid, int sig);
int execlp(const char *file, const char *arg, ... /*, (char *) NULL */);

/* threads, mutex et sémaphores */
int pthread_create(pthread_t *thread, NULL,
                  void *(*start_routine) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_detach(pthread_t thread);

int pthread_mutex_init(pthread_mutex_t *mutex, NULL);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);

int sem_init(sem_t *sem, 0, unsigned int valeur);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);

/* entrées-sorties bas niveau */
int open(const char *path, int oflag, ...);
ssize_t read(int fildes, void *buf, size_t nbyte);
ssize_t write(int fildes, const void *buf, size_t nbyte);
int close(int fildes);
int dup2(int oldfd, int newfd);
int pipe(int fildes[2]);

int poll(struct pollfd *fds, nfds_t nfd, int timeout);
struct pollfd {
    int fd;           /* Descripteur de fichier */
    short events;     /* Événements attendus */
    short revents;    /* Événements détectés */
};

```

```

};

/* sockets et réseau */
int socket(int domain, int type, 0);
int bind(int socket, const struct sockaddr *address, socklen_t address_len);
int listen(int socket, int backlog);
int accept(int socket, struct sockaddr *address, socklen_t *address_len);
int connect(int socket, const struct sockaddr *address, socklen_t address_len);

ssize_t recv(int socket, void *buffer, size_t length, int flags);
ssize_t send(int socket, const void *buffer, size_t length, int flags);
ssize_t recvfrom(int socket, void *buffer, size_t length, int flags,
                 struct sockaddr *address, socklen_t *address_len);
ssize_t sendto(int socket, const void *message, size_t length, int flags,
               const struct sockaddr *dest_addr, socklen_t dest_len);

uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);

const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
int inet_pton(int af, const char *src, void *dst);

struct sockaddr_in {
    sa_family_t    sin_family; /* address family: AF_INET */
    in_port_t      sin_port;   /* port in network byte order */
    struct in_addr  sin_addr;   /* internet address */
};

struct in_addr {
    in_addr_t s_addr;
};

struct addrinfo {
    int             ai_flags;
    int             ai_family;
    int             ai_socktype;
    int             ai_protocol;
    socklen_t       ai_addrlen;
    struct sockaddr *ai_addr;
    char            *ai_canonname;
    struct addrinfo *ai_next;
};

int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints, struct addrinfo **res);
void freeaddrinfo(struct addrinfo *res);
const char *gai_strerror(int errcode);

```

Quelques constantes symboliques :

- POLLIN, POLLHUP
- AF_UNIX, AF_INET, AF_INET6
- SOCK_DGRAM, SOCK_RAW, SOCK_STREAM

- `INADDR_ANY`, `IN6ADDR_ANY_INIT`
- `INET_ADDRSTRLEN`, `INET6_ADDRSTRLEN`
- `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREAT`, `O_APPEND`, `O_TRUNC`